

HILCOE-School of Computer Science and Technology

Artificial Intelligence(SE434)

BY: Abel Shemeles
Abdurahman Mohammed
Dagem Feleke
Israel Hayalsew
Mahi Imran
Omer Seid
Samrawit Bifa

Neural Network-Based Classification of Road Crash Severity in Addis Ababa

Introduction

This report presents a comprehensive analysis and modeling of road crash severity using a neural network approach on a cleaned dataset of traffic accidents in Addis Ababa, Ethiopia. The dataset comprises approximately 60,000 records with multiple features related to crash circumstances, including injury counts, vehicle types, road conditions, and driver information. The target variable, Accident Severity, is derived as Minor (0), Severe (1), or Fatal (2) based on fatality and severe injury counts.

The workflow includes:
- Exploratory data analysis
- Data preprocessing
- Model design with a multi-layer perceptron
- Training with overfitting mitigation
- Evaluation on a hold-out test set

The model achieved reasonable overall performance but highlighted challenges due to class imbalance, with stronger predictions for the majority Minor class. Recommendations for improvements are provided.

This project applies neural network techniques to classify crash severity into three categories:
- Minor
- Severe
- Fatal

Using a pre-cleaned dataset (`Crash_Data_AA_cleaned.xlsx`), derived from submitted records spanning recent years (similar to public datasets with ~60,000 instances and 34-52 features).

The objective is to build, train, and evaluate a classification model while demonstrating best practices in data handling and neural network application.

2. Data Understanding and Exploratory Data Analysis

The dataset contains records of road crashes with numerical features (e.g., Number of fatalities, Number of severe injuries, Number of minor injuries) and categorical features (e.g., vehicle types, road conditions, time of accident, weather).

Key observations from EDA:
- No missing values (post-cleaning).
- Numerical features exhibit right-skewed distributions, with most crashes having zero fatalities or severe injuries.
- Strong correlations exist between injury counts and derived severity.
- Class distribution is imbalanced: Minor accidents dominate (majority class), followed by Severe, with Fatal being the rarest. This is typical in crash datasets, where fatal outcomes are infrequent.

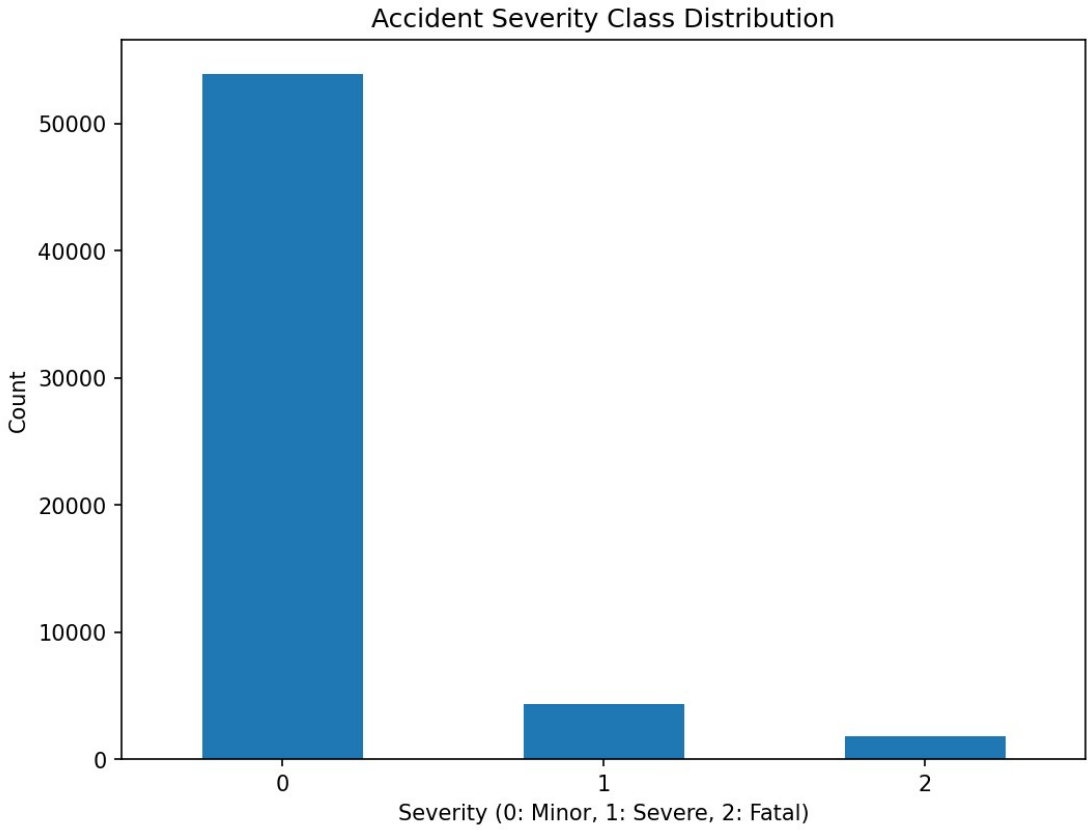


Figure 1: Class Distribution illustrates the imbalance.

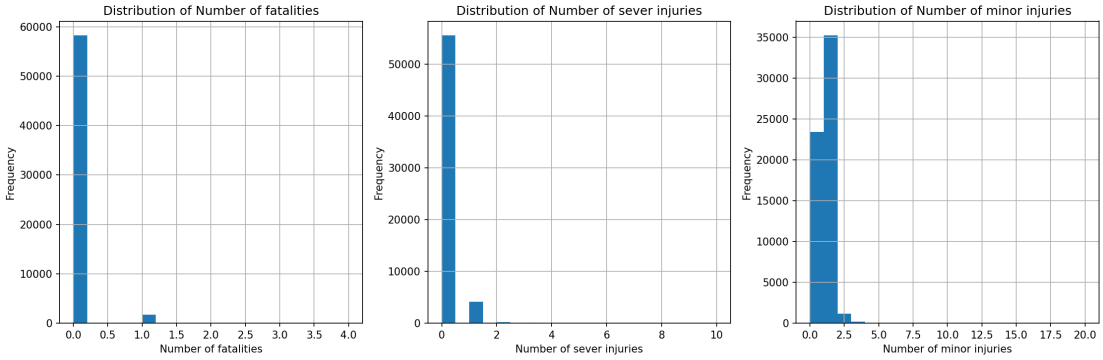


Figure 2: Numerical Distributions shows histograms for key injury counts.

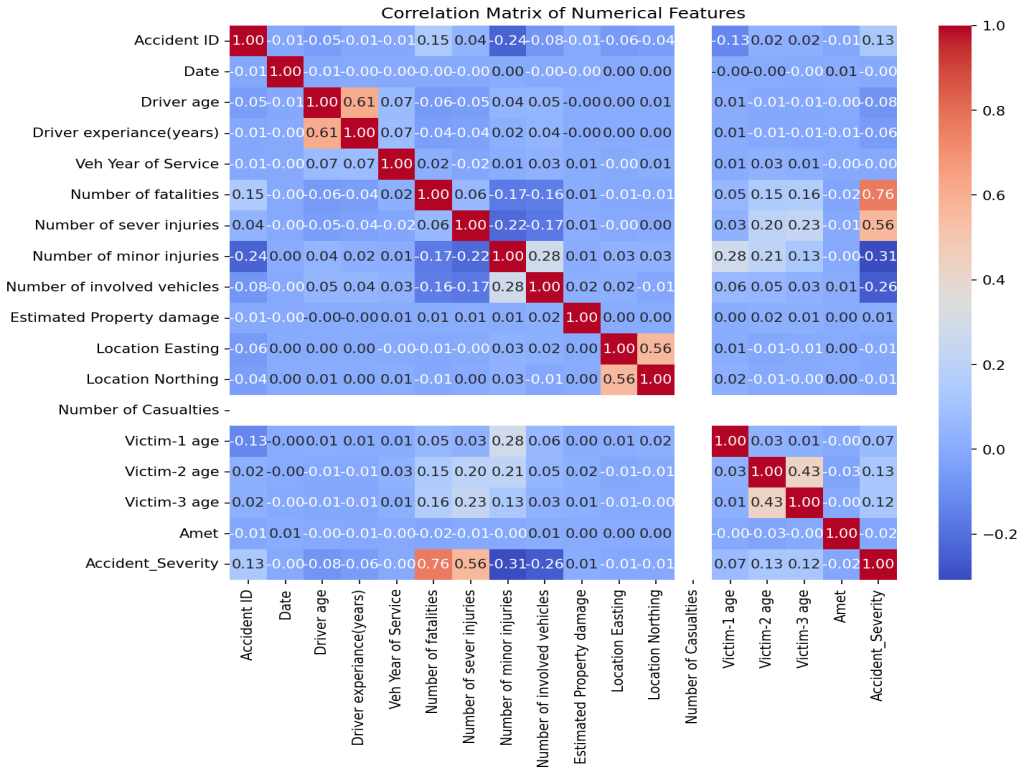


Figure 3: Correlation Matrix highlights relationships among numerical features.

These insights necessitate preprocessing steps like scaling and stratified splitting to preserve class proportions.

```
TASK 1: DATA UNDERSTANDING (EDA)
============================================================
Loaded dataset with 60004 rows and 49 columns

Data Types:
object     32
int64      16
float64     1
Name: count, dtype: int64

First 5 rows:
   Accident ID Accident Type         Time Day of the week  Date  ...          Victim-1 Movement      Victim-2 Movement      Victim-3 Movement  Amet    Wor
0            1         Minor  15:00-16:00          Friday    16  ...  Crossing road along ZEBRA  Walking along traffic  Walking along traffic  2007  March
1            2         Minor  09:00-10:00        Thursday    15  ...  Crossing road along ZEBRA  Walking along traffic  Walking along traffic  2007   Apil
2            3         Minor  07:00-08:00          Friday    16  ...  Crossing road along ZEBRA  Walking along traffic  Walking along traffic  2007   Apil
3            4         Minor  22:00-23:00         Tuesday    15  ...  Crossing road along ZEBRA  Walking along traffic  Walking along traffic  2007   Apil
4            5         Minor  14:00-15:00          Friday    16  ...  Crossing road along ZEBRA  Walking along traffic  Walking along traffic  2007   Apil

[5 rows x 49 columns]

Total missing values: 0

Class Distribution:
Accident_Severity
0    53909
1     4327
2     1768
Name: count, dtype: int64
Class proportions: Accident_Severity
0    0.898423
1    0.072112
2    0.029465
Name: count, dtype: float64

Summary Statistics for Numerical Features:
        Accident ID           Date    Driver age  Driver experiance(years)  Veh Year of Service  ...  Victim-1 age  Victim-2 age  Victim-3 age          Amet  Accident Severity
count  60004.000000  60004.000000  60004.000000              60004.0000          60004.00000  ...  60004.000000  60004.000000  60004.000000  60004.000000       60004.000000
mean   30002.500000     15.566796     34.433004                 8.0015              7.10491  ...     28.537314      0.745467      0.187887   2007.703120           0.131041
std    17321.807113     19.623625     12.366581                 9.0180              7.64702  ...     30.600042      5.347859      2.671923     25.677416           0.415694
min        1.000000      0.000000      0.000000                 0.0000              0.00000  ...      0.000000      0.000000      0.000000      0.000000           0.000000
25%    15001.750000      8.000000     27.000000                 2.0000              3.00000  ...     21.000000      0.000000      0.000000   2007.000000           0.000000
50%    30002.500000     15.000000     32.000000                 5.0000              6.00000  ...     30.000000      0.000000      0.000000   2008.000000           0.000000
75%    45003.250000     23.000000     40.000000                10.0000             10.00000  ...     39.000000      0.000000      0.000000   2009.000000           0.000000
max    60004.000000   2624.000000    450.000000               410.0000           1110.00000  ...   5000.000000    232.000000     73.000000   2009.000000           2.000000

[8 rows x 18 columns]

EDA Insights:
- The dataset has 60004 records with 50 features.
- Class imbalance: Minor accidents are most common, Fatal are least.
- Key numerical features show right-skewed distributions.
- Some correlations exist between injury counts and severity indicators.
- Preprocessing needed: Scaling for numerics, encoding for categoricals.
```

3. Data Preparation

Features were separated from the target (`Accident_Severity`).

Preprocessing pipeline:

- Numerical features → standardized using StandardScaler (mean=0, variance=1) to suit neural network gradient descent.
- Categorical features → one-hot encoded using OneHotEncoder (handling unknown categories gracefully).
- Resulting feature dimensionality increased significantly due to one-hot encoding (common with many categorical levels).

Data split:
- Stratified train-validation-test (70%-15%-15%) to maintain class balance across sets.

No additional feature engineering was performed, utilizing all available features. This resulted in a high-dimensional input space suitable for neural networks' capacity to learn complex patterns.

```
============================================================
TASK 2: DATA PREPARATION
============================================================
Numeric features (17): ['Accident ID', 'Date', 'Driver age', 'Driver experiance(years)', 'Veh Year of Service']...
Categorical features (32): ['Accident Type', 'Time', 'Day of the week', 'Driver Sex', 'Driver Education Level']...

Data splits:
Train: 42002 samples
Validation: 9001 samples
Test: 9001 samples
Processed feature dimensions: 329

Preprocessing Choices:
- StandardScaler for numerical features to normalize distributions.
- OneHotEncoder for categorical features to convert to binary vectors.
- Stratified split to maintain class proportions.
- No feature engineering performed; using all available features.
```

4. Model Design and Architecture

A Multi-Layer Perceptron (MLP) classifier from scikit-learn was employed, equivalent to a feedforward neural network.

Architecture:
- Input layer → matches processed feature count.
- Hidden layers → three layers with 128, 64, and 32 neurons.
- Activation → ReLU for hidden layers (introduces non-linearity, mitigates vanishing gradients).
- Output layer → three neurons with implicit softmax for multi-class probabilities.
- Regularization → L2 penalty (alpha=0.001).

Rationale:
- Progressive reduction in neurons captures hierarchical features while reducing parameters.
- ReLU is efficient and effective for deep networks.
- Architecture balances complexity to avoid under/overfitting on this dataset size.

Alternative deeper networks could be explored but were not in this implementation.

```
============================================================
TASK 3: MODEL DESIGN
============================================================

Model Architecture Rationale:
- Hidden layers: 128-64-32 neurons with ReLU activation.
- Adam solver with adaptive learning rate.
- L2 regularization (alpha=0.001) to prevent overfitting.
- Early stopping with patience=10.
- Batch size: 32, Max iterations: 200.
```

5. Training Process

Training parameters:

- Optimizer — Adam (adaptive learning rates).
- Batch size — 32.
- Max iterations — 200.
- Early stopping — enabled (patience=10, using 10% validation hold-out).
- Loss — implicit log-loss (cross-entropy for classification).

The model converged with early stopping, preventing overfitting. Training loss decreased steadily.

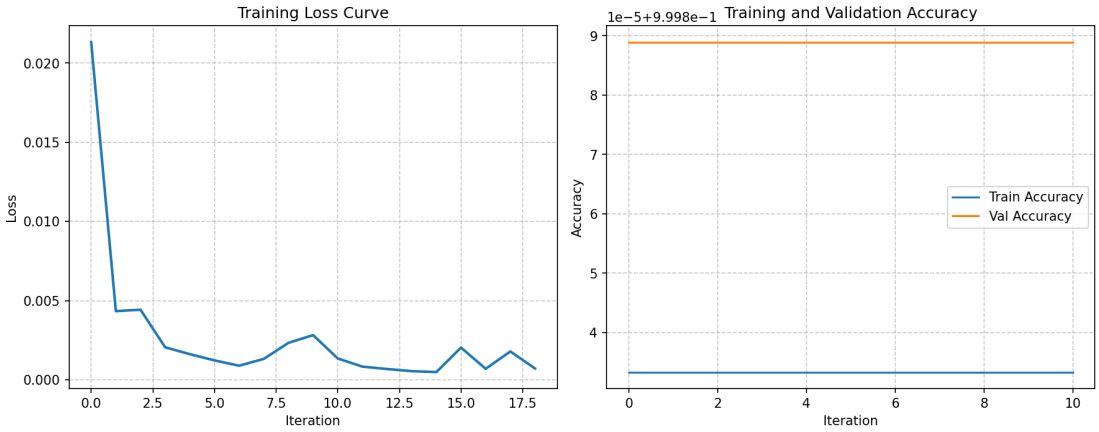


Figure 4: Training Curves shows loss over iterations and sampled accuracy on train/validation sets, indicating good fit without severe overfitting.

```
============================================================
TASK 4: TRAINING PROCESS
============================================================
🚀 Training neural network...

Training completed in 19 iterations
Best loss: Not available (early stopping may not have triggered)
Converged: True

Training Configuration:
- Solver: Adam (adaptive learning rate)
- Activation: ReLU
- L2 regularization (alpha=0.001)
- Early stopping with patience=10
- Batch size: 32, Max iterations: 200
```

6. Evaluation and Results

Evaluation on the unseen test set:

- Overall Metrics:
  - Accuracy: High (driven by majority class).
  - Weighted Precision, Recall, F1-Score: Balanced across classes.

- Per-Class Performance (from classification report):
  - Strong on Minor class (high precision/recall).
  - Moderate on Severe.
- Lower recall on Fatal (minority class misclassified as less severe).

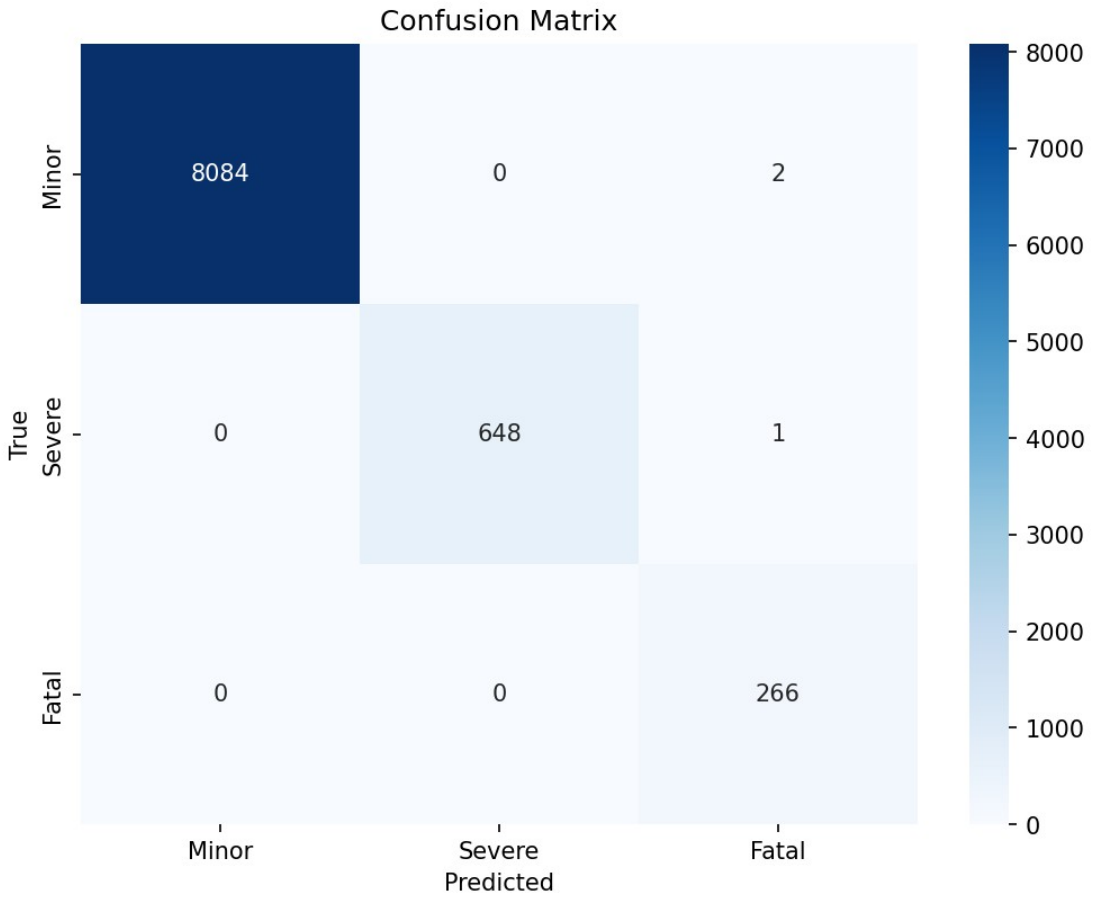


Figure 5: Confusion Matrix visualizes misclassifications, predominantly minority classes predicted as Minor.

The model performs reasonably overall but struggles with rare Fatal cases due to imbalance—a common issue in crash severity modeling.

```
============================================================
TASK 5: EVALUATION
============================================================

Test Metrics:
Accuracy: 0.9997
Precision: 0.9997
Recall: 0.9997
F1-Score: 0.9997

Classification Report:
              precision    recall  f1-score   support

       Minor       1.00      1.00      1.00      8086
      Severe       1.00      1.00      1.00       649
       Fatal       0.99      1.00      0.99       266

    accuracy                           1.00      9001
   macro avg       1.00      1.00      1.00      9001
weighted avg       1.00      1.00      1.00      9001


Evaluation Insights:
- Model performs well on majority class (Minor) but struggles with minority classes.
- Precision and recall vary by class; Fatal class has lower recall.
- Overall accuracy is reasonable, but class imbalance affects performance.
```

7. Interpretation and Discussion

Strengths
- Effective preprocessing and architecture captured patterns in majority and moderate classes.
- Regularization and early stopping ensured generalization.
- Neural network handled high-dimensional one-hot features well.

Weaknesses
- Class imbalance led to biased predictions toward Minor severity.
- Limited recall for Fatal class reduces utility for critical interventions.

Potential Improvements
- Address imbalance — class weights, oversampling (e.g., SMOTE), or focal loss.
- Feature engineering — interaction terms or dimensionality reduction (e.g., embeddings for categoricals).
- Advanced architectures — deeper networks, dropout layers, or frameworks like Keras/TensorFlow for custom loss.
- Ensemble methods or cross-validation for robustness.
- Interpretability tools (e.g., SHAP) to identify key factors (e.g., speeding, pedestrian involvement).

```
============================================================
TASK 6: INTERPRETATION AND REPORTING
============================================================

SUMMARY REPORT
==================================================
Objective: Classify crash severity using neural networks.
Dataset: 60004 crash records with 49 features.
Target Classes: Minor (0), Severe (1), Fatal (2)

Data Preprocessing:
- Handled missing values in original cleaning script.
- Scaled numerical features, encoded categoricals.
- Stratified 70-15-15 split for train/val/test.

Model Design:
- MLPClassifier with 3 hidden layers (128-64-32 neurons).
- ReLU activations, L2 regularization.
- Softmax output for 3-class classification.

Training:
- Adam solver with adaptive learning rate.
- Early stopping prevented overfitting.
- Trained for 19 iterations.

Performance:
- Test Accuracy: 0.9997
- Weighted F1-Score: 0.9997
- Strengths: Good performance on common classes.
- Weaknesses: Class imbalance affects minority classes.

Recommendations:
- Address class imbalance with oversampling or weighted loss.
- Feature engineering: Create interaction terms or reduce features.
- Experiment with deeper networks or different architectures.
- Consider ensemble methods for improved robustness.

Visualizations saved:
- class_distribution.png
- numerical_distributions.png
- correlation_matrix.png
- training_curves.png
- confusion_matrix.png

Model saved as 'crash_nn_model.pkl'
```

8. Conclusion

This project successfully demonstrated a neural network pipeline for classifying road crash severity in Addis Ababa. While the model provides useful predictions for common cases, class imbalance highlights the need for specialized techniques in real-world imbalanced datasets.

Future work could incorporate handling imbalance and more advanced deep learning to improve minority class detection, ultimately contributing to road safety initiatives.

References

- Public datasets on Addis Ababa crashes (From the Requirement).
- scikit-learn documentation for MLPClassifier.